

Django With Python

By Phitron (Week 2)

⇒ Model and Model-form in Django

➤ Creating Model in Django

Command: (makemigrations)

“py manage.py makemigrations”

```
sixth_project > first_app > models.py > Student
1  from django.db import models
2
3  # Create your models here.
4
5  class Student(models.Model):
6      name = models.CharField(max_length=20)
7      roll = models.IntegerField(primary_key=True)
8      address = models.TextField()
```

```
● PS D:\Phitron\Django\sixth_project> py manage.py makemigrations
Migrations for 'first_app':
  first_app\migrations\0001_initial.py
  - Create model Student
```

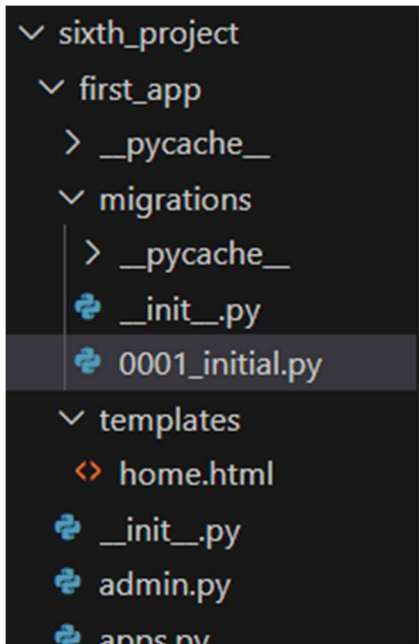
After making migrations we have to migrate:

Command: (migrate)

“py manage.py migrate”

```
● PS D:\Phitron\Django\sixth_project> py manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, first_app, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
```

After first migration the view:



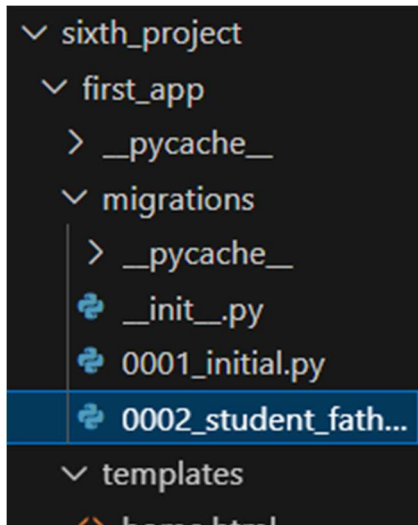
```
sixth_project > first_app > migrations > 0001_initial.py > Migration
1  # Generated by Django 5.0.4 on 2024-04-16 11:20
2
3  from django.db import migrations, models
4
5
6  class Migration(migrations.Migration):
7
8      initial = True
9
10     dependencies = [
11     ]
12
13     operations = [
14         migrations.CreateModel(
15             name='Student',
16             fields=[
17                 ('name', models.CharField(max_length=20)),
18                 ('roll', models.IntegerField(primary_key=True, serialize=False)),
19                 ('address', models.TextField()),
20             ],
21         ),
22     ]
```

- Now if we make any changes to the models.py file such as adding any new field. Then we have to make migration again.

```
sixth_project > first_app > models.py > Student
1  from django.db import models
2
3  # Create your models here.
4
5  class Student(models.Model):
6      name = models.CharField(max_length=20)
7      roll = models.IntegerField(primary_key=True)
8      address = models.TextField()
9      father_name = models.TextField(default="Towsif")
```

```
● PS D:\Phitron\Django\sixth_project> py manage.py makemigrations
Migrations for 'first_app':
  first_app\migrations\0002_student_father_name.py
  - Add field father_name to student
```

Now let's see the changes in migrations:



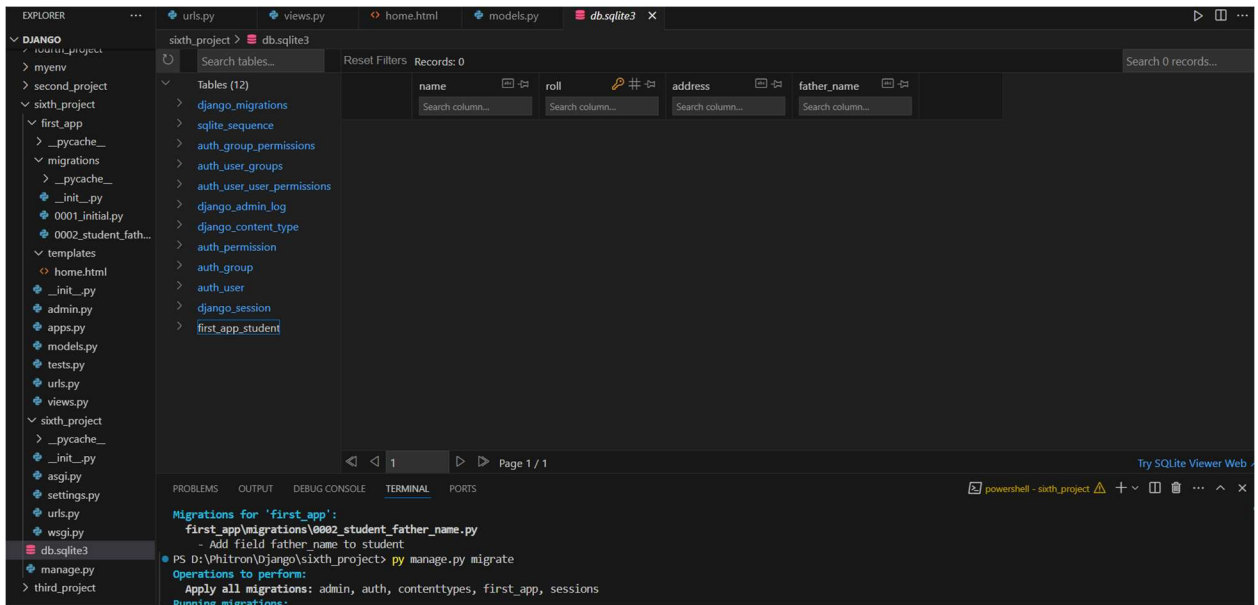
```
sixth_project > first_app > migrations > 0002_student_father_name.py > ...
1  # Generated by Django 5.0.4 on 2024-04-16 11:29
2
3  from django.db import migrations, models
4
5
6  class Migration(migrations.Migration):
7
8      dependencies = [
9          ('first_app', '0001_initial'),
10     ]
11
12     operations = [
13         migrations.AddField(
14             model_name='student',
15             name='father_name',
16             field=models.TextField(default='Towsif'),
17         ),
18     ]
19
```

```
● PS D:\Phitron\Django\sixth_project> py manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, first_app, sessions
Running migrations:
  Applying first_app.0002_student_father_name... OK
```

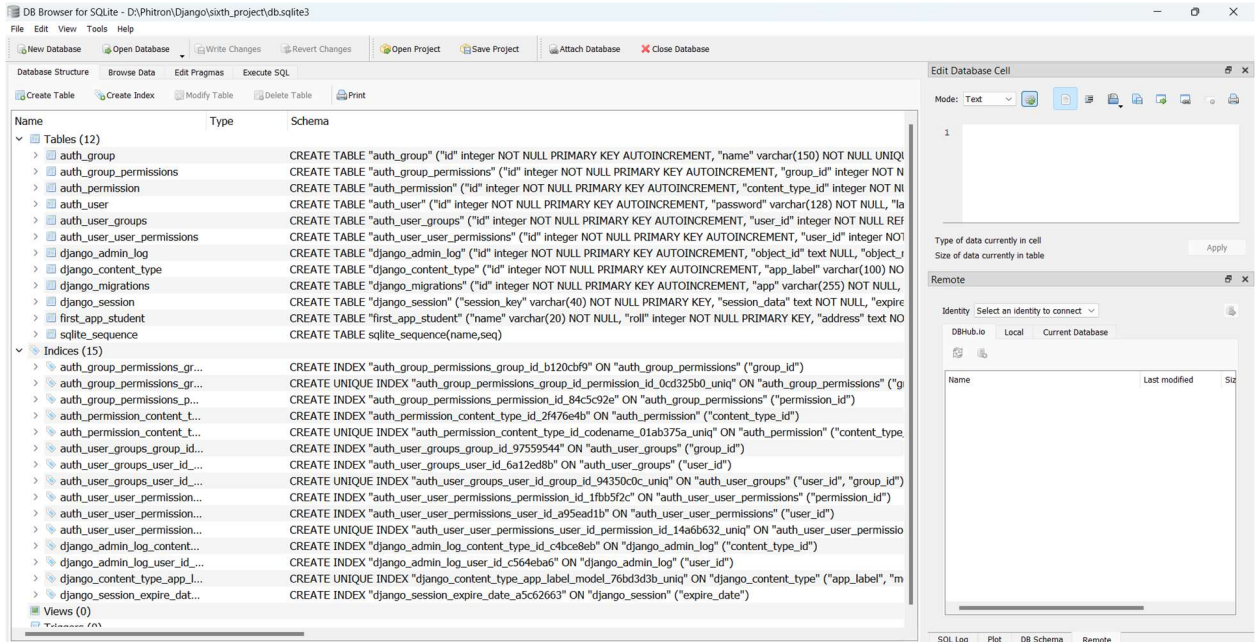
⇒ Showing Model using VS Code and DB Browser

To do so: We have to download two extensions in vs code and download DB Browser SQLite.

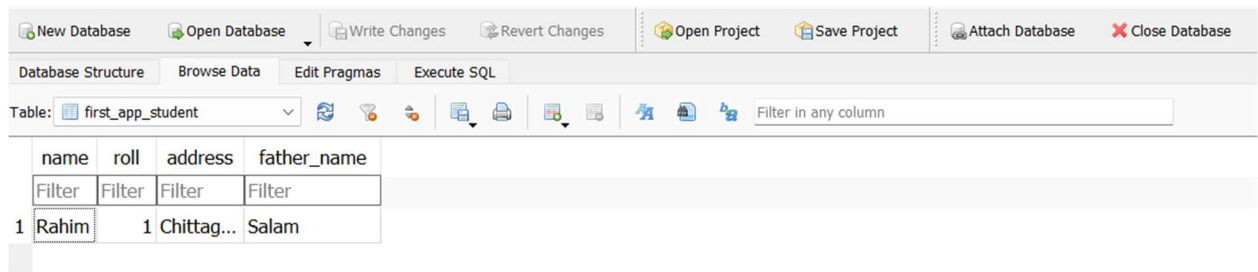
Let's see the view in VS Code:



Now let's see the view in DB Browser:

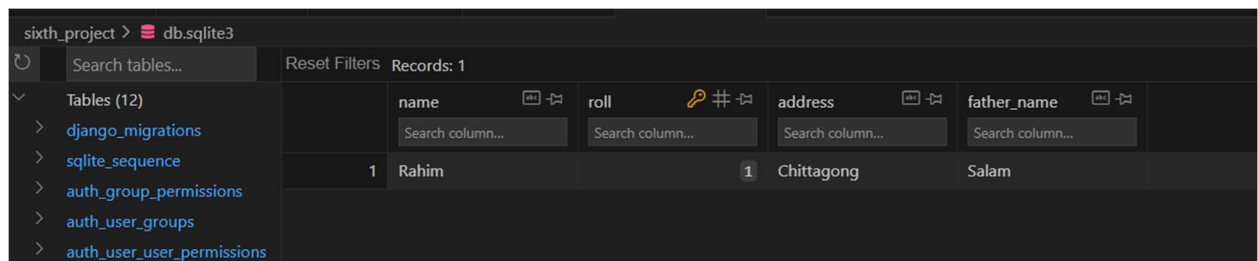


Now if we want to add any records to the database from DB Browser:



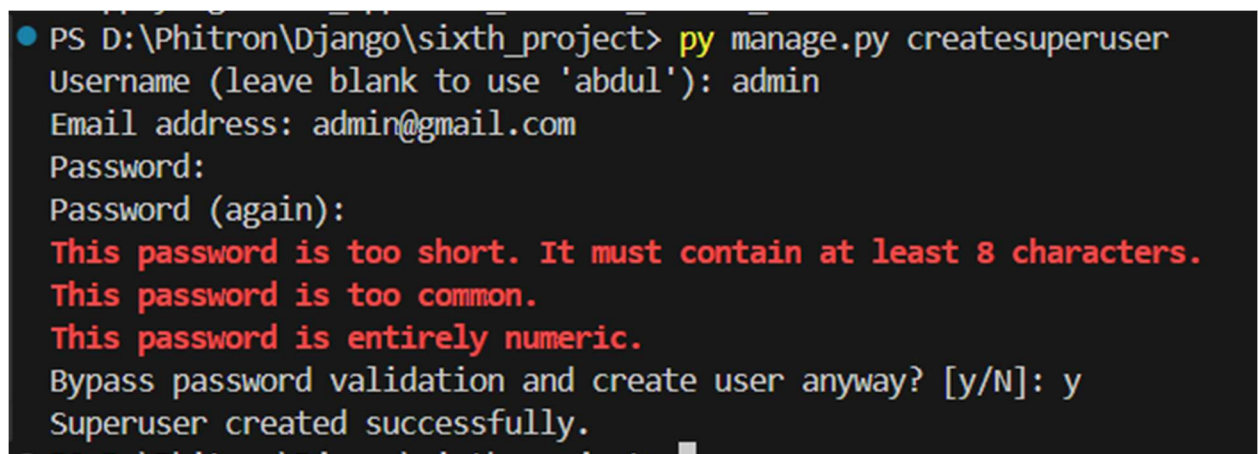
After inserting all data click “write changes”.

Now let's see the vs code:



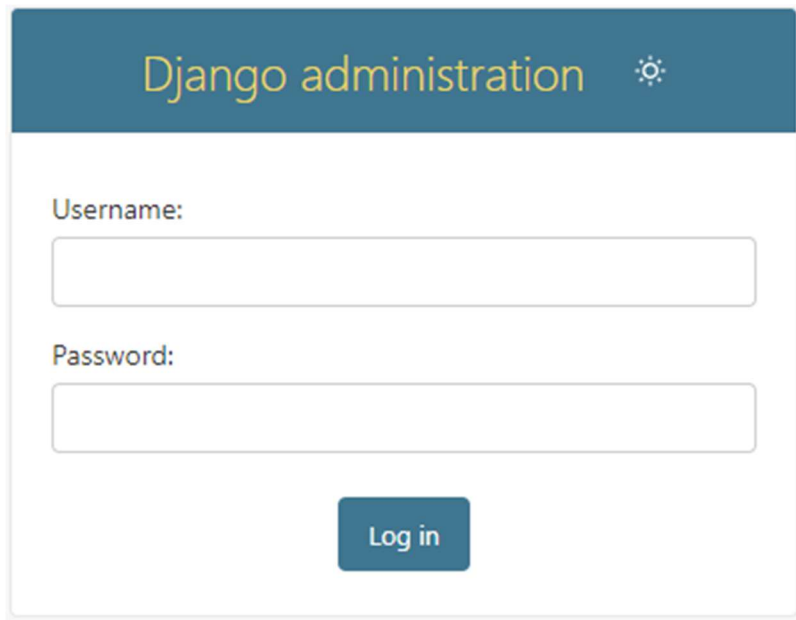
⇒ Creating Superuser and Accessing Admin Panel

Command: “py manage.py createsuperuser”



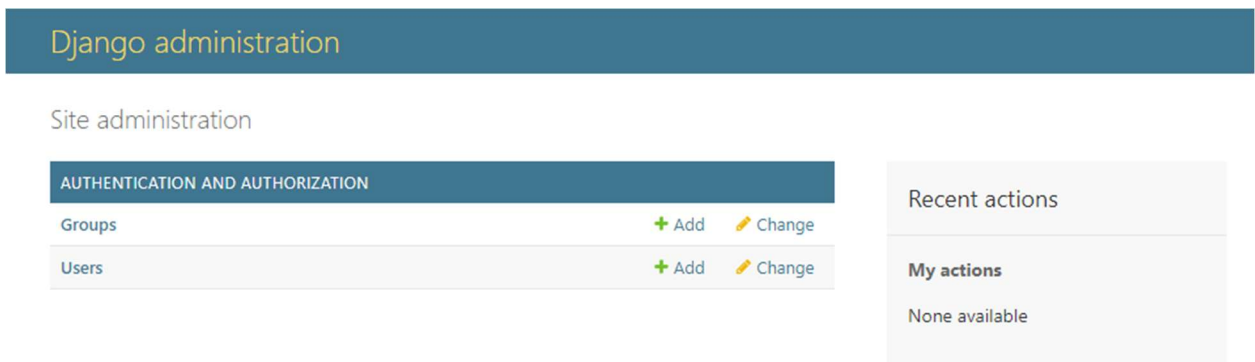
➤ To go to admin panel:





The image shows the Django administration login page. It has a dark blue header with the text "Django administration" and a gear icon. Below the header, there are two input fields: "Username:" and "Password:". Below the password field is a blue "Log in" button.

After first logging in to admin panel:



The image shows the Django administration dashboard after logging in. It has a dark blue header with the text "Django administration". Below the header, there is a section titled "Site administration". Under this section, there is a table with two rows: "Groups" and "Users". Each row has a green "+ Add" button and a yellow pencil icon with the text "Change". To the right of the table, there is a section titled "Recent actions" with a sub-section titled "My actions" which says "None available".

➤ To show the database in the admin panel we have to follow these criteria:

Step 1: We have to register it to first_app/admin.py

```
sixth_project > first_app > admin.py
1  from django.contrib import admin
2
3  # Register your models here.
4
```


Step 2: We have to import the model in admin.py

```
sixth_project > first_app > admin.py
1  from django.contrib import admin
2  from . import models
3  # Register your models here.
4
```

Step 3: Now let's register the model

```
sixth_project > first_app > admin.py
1  from django.contrib import admin
2  from . import models
3  # Register your models here.
4  ⚡
5  admin.site.register(models.Student)
```

Step 4: Now let's see the admin panel

Start typing to filter...	
AUTHENTICATION AND AUTHORIZATION	
Groups	+ Add
Users	+ Add
FIRST_APP	
Students	+ Add

Now it shows us the student model. If we dig into students:

✓ The student "Student object (2)" was added successfully.

Select student to change

Action: 0 of 2 selected

- ☐ STUDENT
- ☐ Student object (2)
- ☐ Student object (1)

2 students

➤ Customizing the Student object

Note: If we add any new model then migrations needed.
Otherwise, it is not necessary.

```
sixth_project > first_app > models.py > Student > __str__
1  from django.db import models
2
3  # Create your models here.
4
5  class Student(models.Model):
6      name = models.CharField(max_length=20)
7      roll = models.IntegerField(primary_key=True)
8      address = models.TextField()
9      father_name = models.TextField(default="Towsif")
10
11     def __str__(self):
12         return self.name
```

Now let's see the admin panel.

Select student to change

Action: 0 of 2 selected

☐	STUDENT
☐	Karim
☐	Rahim

2 students

➤ Showing Model data to frontend

Step 1: In views.py import model

```
sixth_project > first_app > views.py > ...  
1 from django.shortcuts import render  
2 from . import models  
3
```

Step 2: Bring all objects of that model using all.

```
sixth_project > first_app > views.py > home > student  
1 from django.shortcuts import render  
2 from . import models  
3  
4 # Create your views here.  
5 def home(request):  
6     student = models.Student.objects.all()  
7     return render(request, "home.html", {'data' : student})
```

Step 3: Rendering to template.

```
sixth_project > first_app > views.py > home  
1 from django.shortcuts import render  
2 from . import models  
3  
4 # Create your views here.  
5 def home(request):  
6     student = models.Student.objects.all()  
7     return render(request, "home.html", {'data' : student})
```

Step 4: Now accessing in template.

```
sixth_project > first_app > templates > home.html > html > body > table.table-bordered > thead
2   <html lang="en">
12  <body>
13  <!-- THIS IS THE TABLE -->
14  {% if data %}
15    <table class="table table-bordered">
16      <thead>
17        <tr>
18          <th scope="col">Roll</th>
19          <th scope="col">Student name</th>
20          <th scope="col">Father's name</th>
21          <th scope="col">Address</th>
22        </tr>
23      </thead>
24      <tbody>
25        {% for st in data %}
26          <tr>
27            <td>{{st.roll}}</td>
28            <td>{{st.name}}</td>
29            <td>{{st.father_name}}</td>
30            <td>{{st.address}}</td>
31          </tr>
32        {% endfor %}
33      </tbody>
34    </table>
35  {% else %}
36    <h3>No data available</h3>
37  {% endif %}
38  <script src="https://code.jquery.com/jquery-3.2.1.slim.min.js"
39    integrity="sha384-KJ3o2DKtIkvYIK3UENzmM7KCKRr/rE9/Qpg6aAZGJwFDMVNA/GpGFF93hXpG5KkN"
40    crossorigin="anonymous"></script>
41  <script src="https://cdn.jsdelivr.net/npm/popper.js@1.12.9/dist/umd/popper.min.js"
42    integrity="sha384-ApNbgh9B+Y1QKtv3Rn7W3mgPxhU9K/ScQsAP7hUibX39j7fakFPskvXusvfa0b4Q"
43    crossorigin="anonymous"></script>
```

➤ Deleting Model Data from Frontend

Step 1: Define a function in views.py

```
sixth_project > first_app > views.py > delete_student
1   from django.shortcuts import render, redirect
2   from . import models
3
4   # Create your views here.
5   def home(request):
6       student = models.Student.objects.all
7       return render(request, "home.html", {'data' : student})
8
9
10  def delete_student(request, roll):
11      std = models.Student.objects.get(pk = roll).delete()
12      return redirect("homepage")
```

Note: Passed a parameter so that we can track.

Step 2: Defining urls.py

```
sixth_project > first_app > urls.py > ...
1  from django.urls import path
2  from . import views
3
4  urlpatterns = [
5      path('', views.home, name="homepage"),
6      path('delete/<int:roll>', views.delete_student, name="delete_student"),
7  ]
```

Step 3: Declaring the url in template.

```
sixth_project > first_app > templates > home.html > html > body > table.table-bordered > tbody > tr > td
2  <html lang="en">
12 <body>
15     <table class="table table-bordered">
16         <thead>
17             <tr>
19                 <th scope="col">Student name</th>
20                 <th scope="col">Father's name</th>
21                 <th scope="col">Address</th>
22                 <th scope="col">Action</th>
23             </tr>
24         </thead>
25         <tbody>
26             {% for st in data %}
27                 <tr>
28                     <td>{{st.roll}}</td>
29                     <td>{{st.name}}</td>
30                     <td>{{st.father_name}}</td>
31                     <td>{{st.address}}</td>
32                     <td><a href="{% url 'delete_student' st.roll %}" class="btn btn-danger text-white">Delete</a></td>
33                 </tr>
```

Step 4: In views.py add delete () function and import redirect to go through a certain page after delete.

```
sixth_project > first_app > views.py > delete_student
1  from django.shortcuts import render, redirect
2  from . import models
3
4  # Create your views here.
5  def home(request):
6      student = models.Student.objects.all
7      return render(request, "home.html", {'data' : student})
8
9
10 def delete_student(request, roll):
11     std = models.Student.objects.get(pk = roll).delete()
12     return redirect("homepage")
```

